DBMS ASSIGNMENT SERIES

Assignment 7

Sample Data Insertion

Online Food Delivery System

SUBMITTED BY	PROGRAMME & INSTITUTION	SUBMISSION DATE
Vimal Kumar Rao	BCA (AI/ML) Alliance University	24 October 2026

A consistent relational design carried forward from Assignments 1 and 2.

IN THIS ASSIGNMENT

01	02	03	04
Seed records	Respect insert order	Cover statuses	Check row counts

1. Objective and Dataset

This assignment loads fictional but realistic test data into all eight tables. The dataset covers multiple customers, saved addresses, restaurants, menu items, delivery partners, order states, line items, and payment states so later aggregate and join queries return meaningful results.

> **Safety note:** All names, example.com email addresses, and telephone-like values below are synthetic classroom data. Password_Hash values are placeholders only, not usable credentials or production password hashes.

2. Insert Order and Referential Integrity

1. Insert Customer rows first, because Address references Customer.
2. Insert Address, Restaurant, Menu_Item, and Delivery_Partner parent records.
3. Insert Orders after customers, restaurants, addresses, and delivery partners exist.
4. Insert Order_Item rows after Orders and Menu_Item exist.
5. Insert Payment rows after Orders exist.

3. Sample Data Volume

Table	Records inserted	Coverage
Customer	7	Multiple accounts with unique email and phone.
Address	8	One customer has two addresses; all order addresses belong to their customer.
Restaurant	5	Ratings between 4.0 and 4.5.
Menu_Item	15	Three food items per restaurant; prices range from ₹40 to ₹329.
Delivery_Partner	4	Available, Busy, and Offline statuses.
Orders	12	Delivered, Preparing, Out for Delivery, Confirmed, Placed, and Cancelled.
Order_Item	21	Single- and multi-item orders, quantities, and transaction-time prices.
Payment	12	Paid, Pending, Failed, and Refunded states.

4. Example SQL INSERT Statements

The full insert script is provided in Assignment_7_Sample_Data.sql. The excerpt below illustrates the explicit column lists and parent-child insertion approach.

```
INSERT INTO Customer (Customer_ID, Name, Email, Phone, Password_Hash) VALUES
(1, 'Asha Menon', 'asha@example.com', '9000000001', 'demo_hash_customer_01'),
(2, 'Rahul Nair', 'rahul@example.com', '9000000002', 'demo_hash_customer_02');

INSERT INTO Order_Item (Order_ID, Item_ID, Quantity, Unit_Price) VALUES
(1, 1, 2, 220.00),
(1, 3, 1, 150.00);
```

5. Data Validation and Expected Output

The bundled SQL file includes a row-count verification query. Expected counts are as follows:

Table	Expected row count
Customer	7
Address	8
Restaurant	5
Menu_Item	15
Delivery_Partner	4
Orders	12
Order_Item	21
Payment	12

6. Demonstration Queries

```
SELECT COUNT(*) AS Customers FROM Customer;
SELECT COUNT(*) AS Orders FROM Orders;
SELECT Order_ID, Customer_ID, Restaurant_ID, Order_Status
FROM Orders
ORDER BY Order_Date DESC
LIMIT 5;

SELECT o.Order_ID, SUM(oi.Quantity * oi.Unit_Price) AS Line_Subtotal
FROM Orders o
JOIN Order_Item oi ON oi.Order_ID = o.Order_ID
WHERE o.Order_Status <> 'Cancelled'
GROUP BY o.Order_ID
ORDER BY o.Order_ID;
```

Execution note: The data and query outputs were validated against an equivalent local SQLite harness with the same fictional rows. The submitted DDL/DML scripts target MySQL; run them in MySQL Workbench for native result-grid screenshots.

7. Conclusion

The sample dataset populates every table and provides the range of statuses, prices, relationships, and order counts needed for Assignments 8 and 9. Explicit primary/foreign keys and the constraints from Assignment 6 help ensure the data stays relationally consistent.